Cache-Aware Optimisation of Dense Matrix Multiplication and the Density Threshold for Sparse CSR Multiplication

Big Data – Individual Assignment 02

Daniel Rodríguez Alonso
Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria (ULPGC)
Big Data, Academic Year 2026–2027

***Abstract*—This study asks how loop reordering and cache blocking change the runtime of sequential float64 dense matrix multiplication, and at which densities a compressed sparse row (CSR) representation with Gustavson's sparse–sparse product (SpGEMM) beats a dense kernel. All methods are my own C code (gcc 13.3 `-O2`, one thread) on an Intel i7-9750H laptop, validated against NumPy/SciPy and timed with 1 warm-up and $k = 5$ repetitions (median and interquartile range) under a 600 s and 8 GiB per-process budget. Relative to the naive i-j-k loop, reordering (i-k-j) was 2.20–8.37× faster and reordering plus blocking 2.65–12.0× faster (95 % confidence interval at $n = 3072$: [11.9, 12.7]); only the optimised kernels completed $n = 4096$ within budget. At $n = 2048$, SpGEMM was faster than the dense kernel up to 50 % density and slower from 70 %, for uniform and banded nonzeros, a crossover near 55 % that a per-operation cost model reproduces. Conversions dominate at low density, and at equal density the uniform pattern produced up to 13.7× more output nonzeros and ran up to 4.42× slower than the banded one. The conclusions are limited to this machine, compiler and the synthetic inputs tested.**

***Index Terms*—matrix multiplication, cache blocking, loop reordering, sparse matrices, CSR, SpGEMM, benchmarking**

I. Introduction and Objective

Matrix multiplication is a building block of data-intensive computing (linear models, graph analytics, recommendation, neural networks). The classical algorithm performs $2n^3$ floating-point operations for two $n \times n$ matrices, but how fast those operations run depends far more on how the data move through the memory hierarchy than on the operation count itself [1], [2]. When most entries are zero, a sparse representation can avoid both the storage and the arithmetic of the zeros, at the price of index arithmetic, irregular memory access and an output whose sparsity is not known in advance [3], [4].

This study addresses two research questions (RQ) on a single laptop core:

- **RQ-A.** How much do loop reordering and cache blocking reduce the runtime of float64 dense multiplication relative to a validated naive baseline, and how does the effect change with n?
- **RQ-B.** For matched logical inputs, at which tested densities does a compressed sparse row (CSR) representation with Gustavson's sparse–sparse product (SpGEMM) run faster or use less memory than the best dense kernel, and how does the answer depend on the nonzero structure and on the conversion costs?

Throughout, *density* d is the percentage of nonzero entries and *sparsity* $1 - d$ the percentage of zero entries. The hypotheses below were written before the full benchmark (`docs/experiment_plan.md`); they are predictions, not results:

- **H1.** Both optimised kernels are faster than the naive i-j-k baseline, with a gap that grows with n.
- **H2.** Blocking adds a benefit over plain reordering once the three matrices exceed the 12 MiB last-level cache ($n \gtrsim 1024$).
- **H3.** SpGEMM beats the dense kernel below a crossover density that depends on the nonzero pattern; since SpGEMM work grows like $d^2 n^3$ for uniform inputs, the crossover is expected at a relatively high density.
- **H4.** Output fill-in and conversion costs reduce the sparse advantage as d grows; at equal density, a banded pattern is faster than a uniform one.

The comparison is sequential (one thread) and in C for every method. Assignment 1 did not yet provide a validated C kernel when this work started, so an equivalent naive baseline was implemented and validated here (Section III-A). Assignment 1 has since added a C kernel with the same i-j-k loop, scalar accumulator and compiler flags (repository `bigdata-assignment01-matmul-languages`, commit `dfb18bf`); the baseline of this study remains the one validated here.

II. Background and Related Work

Dense kernels and the memory hierarchy. In the i-j-k order each output C_{ij} is a dot product of row i of A and column j of B; with row-major storage, consecutive iterations of the inner loop read B with a stride of $8n$ bytes, so almost every access to B touches a different cache line once B no longer fits in cache [1]. Interchanging the two inner loops (i-k-j) turns the inner loop into an update $C_{i,:} \mathrel{+}= a_{ik} B_{k,:}$ with unit stride on B and C, the same arithmetic in a different order. Such a loop is

a candidate for SIMD vectorisation, which gcc enables at `-O2` since version 12, but only under a "very-cheap" cost model [5]; for the kernels of this study it does not apply (Section III-A). Cache blocking (tiling) partitions the iteration space into $b \times b$ tiles so that the working set of a tile update ($\approx 3 \cdot 8b^2$ bytes) is reused while it stays in cache; Lam et al. showed that the benefit depends strongly on the block size and on cache interference [6]. High-performance libraries combine blocking for every cache level with packing and register-level micro-kernels [2]; this study implements only the first ingredient. The roofline model relates attainable performance to arithmetic intensity and memory bandwidth [7] and is used here only qualitatively. Strassen's algorithm [8] reduces the operation count to $O(n^{2.81})$ but was not implemented (see Section V-A).

Sparse storage and SpGEMM. CSR stores, for each row, the column indices and values of its nonzeros plus a row-pointer array [9]. Gustavson's algorithm [3] computes $C = AB$ row by row: for every nonzero a_{ik} of row i it scatters $a_{ik}B_{k,:}$ into an accumulator and finally gathers the touched columns. Its work is proportional to the number of scalar multiply-adds $\text{flops} = \sum_{a_{ik} \neq 0} \text{nnz}(B_{k,:})$ plus $O(n + \text{nnz}(C))$, which for uniform random inputs is about d^2n^3, i.e. a factor d^2 of the dense work. The output can be much denser than the inputs, and its structure is only known after a symbolic phase [4]. For two independent uniform inputs, $P(C_{ij} \neq 0) \approx 1 - (1 - d^2)^n$, so at $n = 2048$ the product is almost full once $d \gtrsim 5\%$.

Benchmarking. The protocol follows common recommendations: report medians with a dispersion statistic over repeated runs, state the timing boundary and the environment, and treat measurements as sensitive to seemingly irrelevant setup details [10], [11].

III. Methodology

A. Implementations

All kernels are my own C11 code, compiled with gcc 13.3 `-O2 -std=c11` (identical flags, no `-march=native`, no BLAS, one thread), operate on float64 row-major arrays, and share the interface of `src/c/dense/matmul_dense.h`:

- `naive_ijk` (baseline): i-j-k triple loop with a scalar accumulator per C_{ij}.
- `reorder_ikj` (optimisation 1, loop reordering): i-k-j order, C zeroed first; the only change is the loop order, which gives unit stride on B and C and keeps a_{ik} in a register.
- `blocked_ikj`(b) (optimisation 2, cache blocking): the i, k and j loops are tiled into $b \times b$ tiles (edge tiles smaller) with i-k-j inside each tile. The block size is chosen by the study in Section IV-A.
- `csr_gustavson`: own CSR (`int64` row pointers, `int32` column indices, `float64` values; $12\,\text{nnz} + 8(n+1)$ bytes) and own two-phase Gustavson SpGEMM. A symbolic pass counts the distinct columns of every output row with a marker array so that C is allocated exactly; a numeric pass accumulates into a dense accumulator of length n and gathers the touched columns. Output columns are left unsorted; structural nonzeros are kept even if a sum cancels.
- Conversions `dense`→`CSR` (two passes: count, fill) and `CSR`→`dense` (scatter).

TABLE I
Execution environment (captured by `capture_env.sh`, `results/environment.md`).

Item	Value
CPU	Intel Core i7-9750H, 6 cores / 12 threads, 2.6 GHz base, 4.5 GHz max turbo; L1d 32 KiB and L2 256 KiB per core, shared L3 12 MiB; AVX2, FMA
Memory	15 GiB RAM
OS	Ubuntu 24.04.5 LTS, Linux 7.0.0
Power	AC connected, governor `powersave` (intel_pstate), turbo enabled
Compiler	gcc 13.3.0, `-O2 -std=c11`, single thread
Python (analysis, tests)	3.12.3; NumPy 2.5.4, SciPy 1.18.1, pandas 3.1.0, Matplotlib 3.11.2

The generated assembly (`gcc -S`) and the vectoriser report (`-fopt-info-vec-all`) show that none of the three dense kernels is vectorised at `-O2`: all use scalar `mulsd`/`addsd` instructions (no FMA). The dense comparison therefore isolates the effect of the memory-access order; it does not include any SIMD effect. NumPy [12] and SciPy [13] are used only to generate inputs and as correctness references; no library code is inside any timed region.

B. Inputs

Dense inputs are $n \times n$ float64 matrices with entries $\sim U[-1, 1)$ drawn with NumPy's PCG64 generator (`default_rng`, A seed 42, B seed 43). Sparse inputs have exactly $\text{nnz} = \text{round}(dn^2)$ distinct positions with values $\sim U[-1, 1)$ (exact zeros redrawn, so every stored entry is nonzero). Two nonzero patterns are used: *uniform* (positions uniform over the matrix) and *banded* (positions uniform inside the narrowest band $|i - j| \leq w$ that can hold nnz entries). Seeds are $1000 + 10^6 d$ for A and that value $+1$ for B. Every input is written once to `data/inputs/` (raw float64, or coordinate-format (COO) triplets) with a JSON sidecar (generator, seed, nnz, band width) and a SHA-256 checksum, and is read by every method, so all methods see byte-identical inputs. The tested densities at $n = 2048$ are $d \in \{0.1, 0.5, 1, 2, 5, 10, 20, 30, 50, 70, 100\}\%$; the sparse scaling study uses $d \in \{0.1, 1\}\%$ and $n \in \{1024, \ldots, 16384\}$, plus $n = 32768$ at $d = 0.1\%$.

C. Experimental environment

Table I summarises the machine. It is a shared laptop: other course benchmarks ran on it during the same period, serialised by a machine-wide lock (Section III-E).

D. Correctness validation

`tests/test_correctness.py` runs the compiled programs and must pass before any benchmark. Dense kernels (and blocked with $b \in \{1, 3, 16, 64, 256\}$, including sizes not

divisible by b) are checked on known 2×2 and 3×3 products, identity and zero matrices (exact equality), and random matrices with $n \leq 257$ against NumPy. Sparse modes are checked against SciPy on identity, zero, empty rows/columns, a known 2×2 case and generated uniform and banded inputs; $\mathrm{nnz}(C)$ must equal the structural nonzero count, and the reported flop count must match an independent count. The floating-point tolerance is the componentwise bound of the classical algorithm evaluated in any order [14], $|C - C_{\mathrm{ref}}|_{ij} \leq 2nu(|A||B|)_{ij}$ with $u = 2^{-53}$ (the factor 2 covers both evaluation orders). All checks passed; over the 74 tolerance comparisons the largest error was 3.2×10^{-14}, at most 0.30 of the bound.

E. Measurement protocol and budgets

Timing boundaries. Time is measured with `clock_gettime(CLOCK_MONOTONIC)` around one kernel call; loading inputs and allocating and zeroing C happen before the timer, and a checksum of C is printed afterwards so that the computation cannot be removed. The boundaries are never mixed in one comparison: `kernel` (Part A); `spgemm_kernel` (CSR×CSR→CSR including the symbolic phase and the allocation of C; inputs already in CSR); `dense_kernel` (blocked kernel on the densified sparse inputs); and the conversions `dense`→`CSR` (both inputs) and `CSR`→`dense` (output), measured separately in their own process. "SpGEMM including conversions" is the explicit sum dense→CSR + SpGEMM + CSR→dense.

Repetitions and statistics. One process per configuration runs 1 warm-up and $k = 5$ measured repetitions; for the costly dense sizes $n \geq 3072$ there is no warm-up and $k = 3$ (declared in advance for the time budget). Warm-ups are excluded; no other rows are excluded. I report the median and the interquartile range (IQR); throughput is reported in GFLOP/s ($2n^3$ floating-point operations divided by the median time, in units of 10^9 per second); speedups are ratios of medians with 95 % bootstrap confidence intervals (CI; 4000 resamples of both samples).

Memory. Two metrics, never mixed: the peak resident set size (RSS) of the whole process (GNU `time` %M), and an analytical array-storage model reported by the program (dense $3 \cdot 8n^2$ bytes; CSR $12\,\mathrm{nnz} + 8(n+1)$ bytes per matrix plus the $12n$-byte accumulator).

Order, lock and budgets. Each configuration runs under a machine-wide lock (`run_measured.py`) so that benchmarks of different assignments never overlap; the method order is shuffled per size with a fixed seed; the 1-minute load average is recorded with every row. Budgets agreed before the full run: at most 600 s per process (exceeding it is recorded as a *timeout*), at most 8 GiB of address space per process, enforced with `prlimit` (an allocation failure is recorded as *oom*), and at most 90 minutes of total machine time. Part A uses $n \in \{256, 512, 768, 1024, 1536, 2048, 3072, 4096\}$; a method stops escalating after its first failure. Block-size rule (declared before running): $b \in \{16, 32, 48, 64, 96, 128, 256\}$ is swept at $n = 1024$ and $n = 2048$, and the fastest median at $n = 2048$ is used for every other dense measurement.

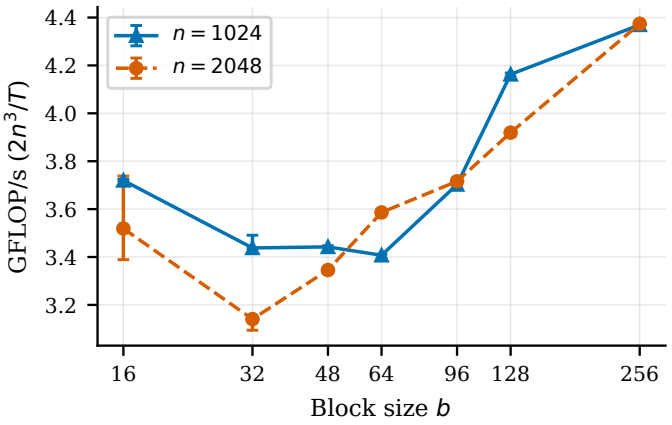


Fig. 1. Block-size study of `blocked_ikj`: GFLOP/s ($2n^3/T$) vs. block size b at $n = 1024$ and $n = 2048$. Points: median; error bars: IQR over $k = 5$ repetitions (1 warm-up excluded).

TABLE II
BLOCK-SIZE STUDY FOR BLOCKED IKJ: MEDIAN KERNEL TIME [IQR] OVER $k = 5$ REPETITIONS (1 WARM-UP EXCLUDED). * = FASTEST; THE RULE SELECTS THE FASTEST b AT $n = 2048$.

b	$n = 1024$: median [IQR]	$n = 2048$: median [IQR]
16	577 ms [782 µs]	4.88 s [473 ms]
32	625 ms [9.97 ms]	5.47 s [89.9 ms]
48	624 ms [1.86 ms]	5.14 s [26.7 ms]
64	630 ms [2.92 ms]	4.79 s [8.05 ms]
96	580 ms [1.92 ms]	4.62 s [14.9 ms]
128	516 ms [1.92 ms]	4.38 s [20.2 ms]
256	491 ms* [1.62 ms]	3.93 s* [25.9 ms]

IV. Results

The full benchmark produced 868 raw rows (725 measured repetitions after excluding warm-ups) from 126 process runs in 49 minutes of locked machine time; one run failed under the budget (Section IV-B). The 1-minute load average at the start of each run had a median of 1.0 (maximum 1.3). Within-configuration variability was small: the relative IQR had a median of 0.66 % (maximum 4.3 %) for the dense kernels and 0.94 % (maximum 34 %, for sub-millisecond SpGEMM runs) in Part B.

A. Block-size study

Figure 1 and Table II show that performance is not monotone in b: at $n = 2048$ the slowest block size ($b = 32$) is 39 % slower than the fastest ($b = 256$, 3.93 s); at $n = 1024$ the spread is 28 %. The declared rule therefore selects $b = 256$, which is also the fastest at $n = 1024$. It is the largest block size tested, so the optimum may lie outside the tested grid.

B. Dense optimisation (Part A)

Table III lists the kernel times. All three methods complete every size up to $n = 3072$; at $n = 4096$, `naive_ijk` exceeded the 600 s process budget (timeout at n=4096), so its largest feasible tested size is $n = 3072$, while both optimised kernels completed $n = 4096$ (46.5 s and 29.7 s). Peak RSS grows from 3 MiB to 386 MiB, in line with the $3 \cdot 8n^2$ bytes

TABLE III

PART A: KERNEL TIME, MEDIAN [IQR] OVER k MEASURED REPETITIONS, AND GFLOP/S $= 2n^3/T_{med}$. BLOCKED IKJ USES $b = 256$. RSS = MEDIAN PEAK RESIDENT SET SIZE [MIB] OF THE PROCESS (GNU TIME), ALL METHODS. $n \le 2048$: 1 WARM-UP + $k = 5$; $n \ge 3072$: NO WARM-UP, $k = 3$.

n	naive: median [IQR]	GF/s	ikj: median [IQR]	GF/s	blocked: median [IQR]	GF/s	k	RSS
256	20.7 ms [58.5 µs]	1.62	8.97 ms [131 µs]	3.74	6.99 ms [10.8 µs]	4.80	5	3.02
512	180 ms [730 µs]	1.49	72.6 ms [714 µs]	3.70	58.9 ms [526 µs]	4.56	5	7.52
768	540 ms [368 µs]	1.68	245 ms [706 µs]	3.70	203 ms [714 µs]	4.45	5	15.0
1024	1.57 s [3.68 ms]	1.36	605 ms [1.74 ms]	3.55	469 ms [1.93 ms]	4.58	5	25.5
1536	9.89 s [131 ms]	0.733	2.48 s [54.4 ms]	2.92	1.58 s [17.9 ms]	4.57	5	55.5
2048	39.2 s [258 ms]	0.438	5.89 s [64.5 ms]	2.91	4.13 s [176 ms]	4.16	5	97.5
3072	170 s [3.51 s]	0.340	20.4 s [191 ms]	2.85	14.1 s [146 ms]	4.10	3	218
4096	timeout (>600 s)		46.5 s [125 ms]	2.96	29.7 s [20.2 ms]	4.63	3	386

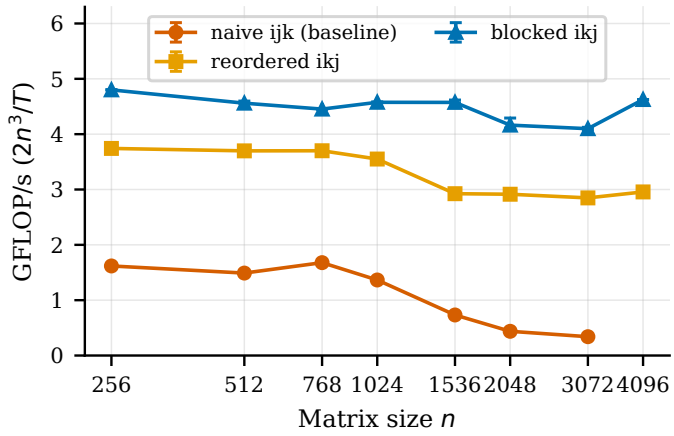


Fig. 2. Part A: GFLOP/s ($2n^3/T_{\text{med}}$) of the three dense kernels vs. n (blocked with $b = 256$). Error bars: IQR. $k = 5$ for $n \le 2048$, $k = 3$ for $n \ge 3072$. The naive kernel has no point at $n = 4096$ (timeout).

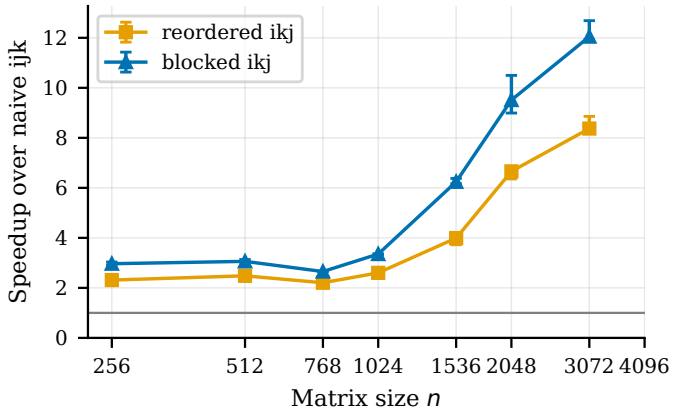


Fig. 3. Speedup over `naive_ijk` (ratio of medians at the same n); error bars: 95 % bootstrap CI. No value at $n = 4096$ because the baseline timed out.

of the three matrices, and is the same for the three methods at a given n.

The throughput of the baseline falls from 1.68 to 0.340 GFLOP/s as n grows, with the main drop between $n = 1024$ and $n = 2048$ (Figure 2); a log–log fit of its median time over $n \ge 512$ gives an exponent of 3.97, well above the cubic operation count. `reorder_ikj` stays at 3.5–

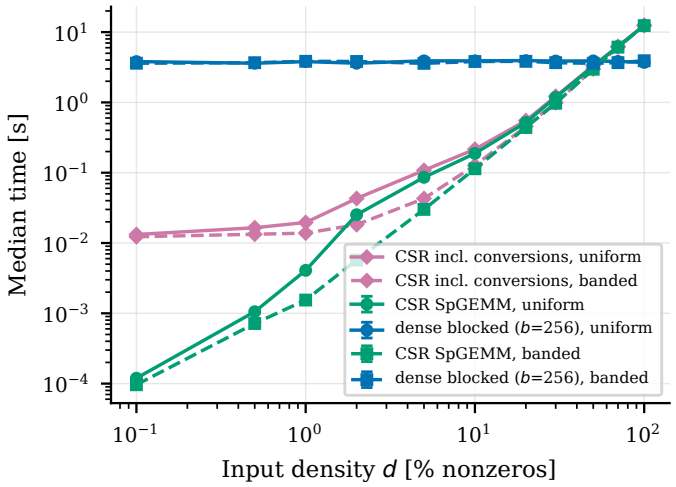


Fig. 4. Part B at $n = 2048$: median time vs. input density (log–log) of CSR SpGEMM, of the dense blocked kernel on the same logical inputs, and of SpGEMM plus both conversions. Solid: uniform pattern; dashed: banded. Error bars: IQR over $k = 5$ repetitions.

3.7 GFLOP/s up to $n = 1024$ and drops to about 2.9 GFLOP/s from $n = 1536$ on (exponent 3.14). `blocked_ikj` stays between 4.10 and 4.80 GFLOP/s at all sizes (exponent 3.02).

Consequently the speedups grow with n (Figure 3): from 2.31× to 8.37× for `reorder_ikj` (95 % CI at $n = 3072$: [8.35, 8.86]) and from 2.97× to 12.0× for `blocked_ikj` (CI [11.9, 12.7]), with a dip at $n = 768$ for both. Blocked over reordered is 1.28× at $n = 256$, between 1.20× and 1.29× up to $n = 1024$, and between 1.43× and 1.56× for $n \ge 1536$ (CI at $n = 4096$: [1.56, 1.57]).

C. *Sparse vs. dense at $n = 2048$ (Part B)*

Table IV and Figure 4 compare the two representations on the same inputs. The dense kernel takes between 3.59 s and 3.93 s at every density, as its work does not depend on the values. SpGEMM time grows with density by five orders of magnitude, from 119 µs (uniform, $d = 0.1\%$) to 12.4 s ($d = 100\%$). SpGEMM is faster than the dense kernel at every tested density up to 50 % for both patterns (dense/SpGEMM ratio at $d = 50\%$: 1.19 uniform, 1.21 banded) and slower at 70 % and 100 % (ratio 0.298 at 100 %). Interpolating between the

TABLE IV

PART B AT $n = 2048$: MEDIAN TIME [IQR] OVER $k = 5$ REPETITIONS (1 WARM-UP EXCLUDED) OF CSR SPGEMM AND OF THE DENSE BLOCKED KERNEL ON THE SAME LOGICAL INPUTS; NNZ(B) = NNZ(A). T_D/T_S: DENSE MEDIAN OVER SPGEMM MEDIAN (> 1: SPARSE FASTER). CONVERSIONS (MEDIANS) ARE MEASURED SEPARATELY; DENSE→CSR CONVERTS BOTH INPUTS. T_S^{conv} = DENSE→CSR + SPGEMM + CSR→DENSE. RSS: MEDIAN PEAK RESIDENT SET OF THE SPGEMM PROCESS / OF THE DENSE PROCESS.

d [%]	nnz(A)	nnz(C)	SpGEMM [IQR]	dense [IQR]	T_D/T_S	dense→CSR	CSR→dense	T_D/T_S^{conv}	RSS S/D [MiB]
uniform pattern									
0.10	4 194	8 728	119 µs [3.12 µs]	3.81 s [36.8 ms]	31937	11.4 ms	1.70 ms	288	2.04 / 97.7
0.50	20 972	209 729	1.05 ms [52.5 µs]	3.61 s [9.24 ms]	3424	13.0 ms	2.39 ms	219	4.84 / 97.8
1.0	41 943	776 711	4.08 ms [39.0 µs]	3.81 s [13.2 ms]	932	11.9 ms	3.57 ms	194	12.1 / 97.8
2.0	83 886	2 346 775	25.3 ms [793 µs]	3.61 s [2.34 ms]	143	13.0 ms	4.75 ms	84.0	31.6 / 97.8
5.0	209 715	4 169 360	85.9 ms [173 µs]	3.90 s [11.4 ms]	45.4	16.0 ms	6.17 ms	36.1	57.3 / 97.8
10	419 430	4 194 304	188 ms [6.67 ms]	3.90 s [18.5 ms]	20.8	21.3 ms	6.29 ms	18.1	65.7 / 97.8
20	838 861	4 194 304	518 ms [366 µs]	3.93 s [12.1 ms]	7.59	27.0 ms	6.00 ms	7.13	81.7 / 97.8
30	1 258 291	4 194 304	1.18 s [34.5 ms]	3.88 s [136 ms]	3.28	32.1 ms	5.90 ms	3.18	81.7 / 97.8
50	2 097 152	4 194 304	3.27 s [14.8 ms]	3.89 s [6.12 ms]	1.19	47.7 ms	5.58 ms	1.17	114 / 97.8
70	2 936 013	4 194 304	6.22 s [11.8 ms]	3.82 s [24.9 ms]	0.615	30.9 ms	5.59 ms	0.611	146 / 110
100	4 194 304	4 194 304	12.4 s [309 ms]	3.70 s [63.6 ms]	0.298	46.9 ms	5.49 ms	0.297	162 / 129
banded pattern									
0.10	4 194	6 562	96.3 µs [17.8 µs]	3.61 s [10.1 ms]	37470	11.1 ms	1.08 ms	294	1.91 / 97.7
0.50	20 972	42 270	716 µs [18.1 µs]	3.67 s [10.9 ms]	5127	11.4 ms	1.17 ms	276	2.78 / 97.8
1.0	41 943	83 335	1.54 ms [48.0 µs]	3.84 s [48.6 ms]	2490	11.1 ms	1.18 ms	278	3.89 / 97.6
2.0	83 886	171 892	5.72 ms [193 µs]	3.82 s [149 ms]	668	11.2 ms	1.28 ms	210	6.11 / 97.8
5.0	209 715	417 032	30.1 ms [428 µs]	3.59 s [13.2 ms]	120	11.3 ms	1.57 ms	83.8	12.8 / 97.8
10	419 430	817 865	114 ms [612 µs]	3.82 s [7.68 ms]	33.5	11.3 ms	1.98 ms	30.0	23.8 / 97.8
20	838 861	1 584 455	440 ms [6.40 ms]	3.84 s [4.93 ms]	8.72	11.9 ms	2.75 ms	8.44	45.8 / 97.8
30	1 258 291	2 296 789	973 ms [8.95 ms]	3.67 s [36.9 ms]	3.78	12.8 ms	3.54 ms	3.71	67.2 / 97.8
50	2 097 152	3 476 046	2.96 s [270 µs]	3.60 s [8.55 ms]	1.21	24.4 ms	4.65 ms	1.20	108 / 97.8
70	2 936 013	4 156 084	6.12 s [44.3 ms]	3.68 s [11.2 ms]	0.601	14.1 ms	5.28 ms	0.599	145 / 110
100	4 194 304	4 194 304	12.4 s [302 ms]	3.92 s [4.74 ms]	0.317	46.7 ms	5.25 ms	0.315	162 / 129

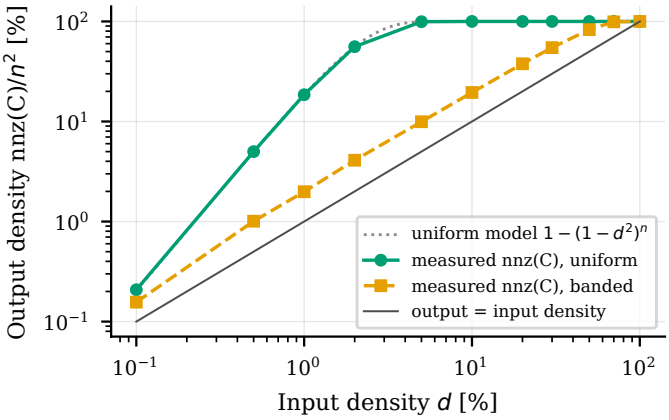


Fig. 5. Output density $\mathrm{nnz}(C)/n^2$ vs. input density at $n = 2048$, with the expectation $1 - (1 - d^2)^n$ for independent uniform inputs and the identity line.

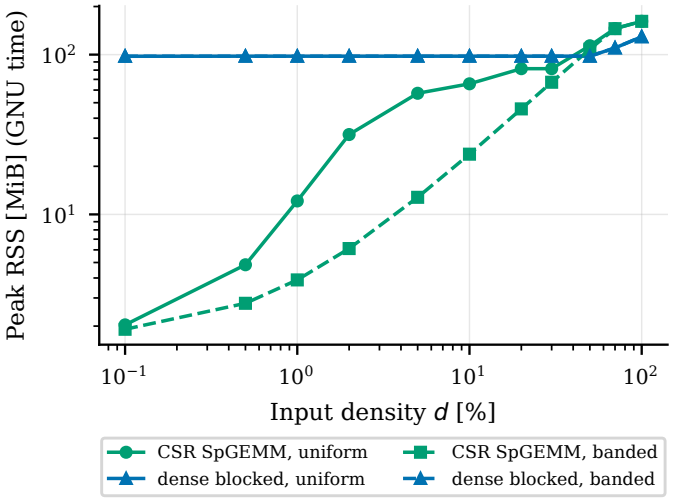


Fig. 6. Peak RSS (GNU `time`) of the SpGEMM process and of the dense process vs. input density at $n = 2048$ (median over the process runs; each process holds its own inputs and output).

bracketing levels places the crossover at about 55 % (uniform) and 56 % (banded).

Conversion costs. Converting both inputs from dense to CSR takes 11–48 ms and converting the output back 1–6 ms (Table IV). They do not move the crossover (55 % including conversions) because they are small against the dense kernel, but they dominate at low density: at $d = 0.1\%$ they are 99 % of the end-to-end sparse time and reduce the advantage over dense from 31937× to 288× (uniform); at $d = 1\%$, from 932× to 194×.

Output fill-in and pattern. Figure 5 shows that the uniform product follows the $1 - (1 - d^2)^n$ expectation and is already 99.4 % full at $d = 5\%$, whereas the banded product stays at most about twice as dense as its inputs. Both patterns have the same number of multiply-adds at each density (uniform/banded ratio 0.96–1.02), yet $\mathrm{nnz}(C)$ is up to 13.7× larger for uniform inputs (at $d = 2\%$), and uniform SpGEMM is up to 4.42× slower (at $d = 2\%$; 2.64× at 1 %). The cost per multiply-add ranges from 1.2 to 14 ns: it is highest at $d = 0.1\%$ (14 ns uniform), stays at 4–7 ns for uniform inputs between 0.5 and

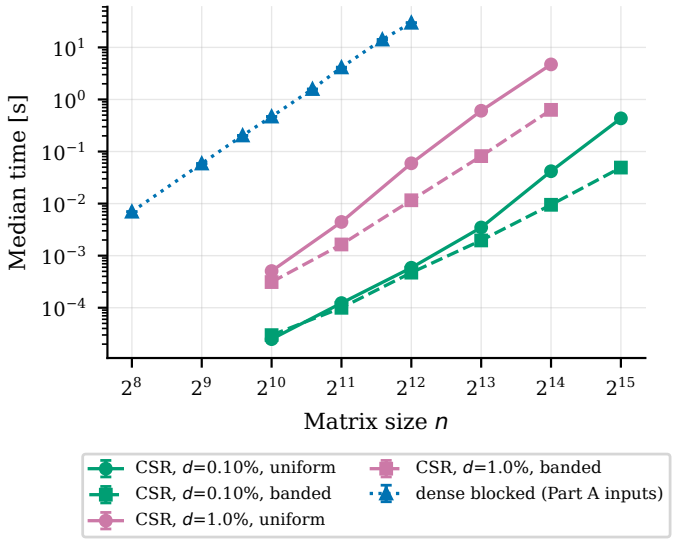


Fig. 7. CSR SpGEMM median time vs. n at fixed density, with the dense blocked kernel of Part A (dense random inputs) for reference. Error bars: IQR over $k = 5$ repetitions.

TABLE V
SPARSE SCALING WITH n AT FIXED DENSITY (CSR SPGEMM ONLY): MEDIAN [IQR] OVER $k = 5$ REPETITIONS (1 WARM-UP EXCLUDED), PEAK RSS OF THE PROCESS (GNU TIME).

d [%]	pattern	n	nnz(A)	nnz(C)	SpGEMM median [IQR]	RSS [MiB]
0.10	uniform	1024	1 049	1 119	24.9 µs [5.41 µs]	1.91
0.10	uniform	2048	4 194	8 728	123 µs [4.49 µs]	1.91
0.10	uniform	4096	16 777	68 536	586 µs [11.8 µs]	3.09
0.10	uniform	8192	67 109	546 076	3.48 ms [63.3 µs]	10.5
0.10	uniform	16384	268 435	4 362 757	41.9 ms [4.05 ms]	62.0
0.10	uniform	32768	1 073 742	34 608 399	434 ms [5.96 ms]	439
0.10	banded	1024	1 049	1 007	29.8 µs [4.57 µs]	1.91
0.10	banded	2048	4 194	6 562	99.7 µs [3.42 µs]	2.04
0.10	banded	4096	16 777	32 885	472 µs [13.5 µs]	2.58
0.10	banded	8192	67 109	135 812	1.96 ms [80.3 µs]	5.34
0.10	banded	16384	268 435	537 939	9.48 ms [176 µs]	16.2
0.10	banded	32768	1 073 742	2 127 957	49.2 ms [935 µs]	59.3
1.0	uniform	1024	10 486	102 430	507 µs [35.2 µs]	3.17
1.0	uniform	2048	41 943	776 711	4.44 ms [200 µs]	12.1
1.0	uniform	4096	167 772	5 638 372	59.4 ms [1.23 ms]	72.6
1.0	uniform	8192	671 089	37 528 559	607 ms [3.42 ms]	457
1.0	uniform	16384	2 684 355	216 279 824	4.71 s [7.24 ms]	2579
1.0	banded	1024	10 486	21 075	311 µs [5.74 µs]	2.15
1.0	banded	2048	41 943	83 335	1.64 ms [13.0 µs]	3.89
1.0	banded	4096	167 772	345 579	11.6 ms [555 µs]	10.7
1.0	banded	8192	671 089	1 344 510	81.7 ms [663 µs]	37.5
1.0	banded	16384	2 684 355	5 362 984	631 ms [1.21 ms]	145

5 %, and converges to 1.2–1.5 ns for both patterns at $d \geq 20\%$. From $d = 70\%$ on the two patterns are indistinguishable (time ratio at most 1.02).

Memory. The SpGEMM process uses less memory than the dense process (97.8 MiB) up to $d = 30\%$ (Figure 6): 12.1 MiB (uniform) and 3.89 MiB (banded) at $d = 1\%$. From $d = 50\%$ on it uses more (114 MiB at 50 %, 162 MiB at 100 %). The analytical storage model gives the same picture (96.0 MiB dense; 144 MiB CSR at $d = 100\%$).

D. Sparse scaling with n

All 22 sparse scaling configurations completed (Figure 7 and Table V). At $d = 0.1\%$ SpGEMM reached $n = 32768$ in 434 ms (uniform, 439 MiB) and 49.2 ms (banded, 59.3 MiB), whereas the three dense float64 matrices at that size would need 24 GiB and were not attempted. At $d = 1\%$ and $n = 16384$, the uniform product has 216 279 824 nonzeros and needed 4.71 s and 2579 MiB, against 631 ms and 145 MiB for the banded product (5 362 984 nonzeros). Fitted growth exponents of the median time over n are 3.35 (uniform) and 2.76 (banded) at $d = 1\%$, and 2.81 and 2.15 at $d = 0.1\%$.

V. DISCUSSION

RQ-A: effect of the dense optimisations. On this machine both optimisations reduce the runtime at every tested size, and the benefit grows with n: 2.20–8.37× for loop reordering and 2.65–12.0× for reordering plus blocking, relative to the naive baseline. H1 is supported. The baseline's throughput collapses once its working set leaves the caches (exponent 3.97 instead of 3), which is consistent with its strided access to B: each multiply-add uses one 8-byte element of a different 64-byte cache line. Up to $n = 1024$ the baseline is still 2.2–2.6× slower than `reorder_ikj`; since none of the kernels is vectorised (Section III-A), a plausible explanation, not verified here, is that the baseline's single scalar accumulator forms a chain of dependent floating-point additions, whereas the i-k-j updates of different C_{ij} are independent.

Separating reordering from blocking (H2). Two effects must be distinguished. First, blocked is already 1.28× faster than reordered at $n = 256$, where $b = 256 = n$ means a single tile and the two kernels execute the same iteration order; this part of the gain cannot be due to cache reuse. The compiled inner loops differ only in address arithmetic (two pointer increments vs. one indexed offset); attributing the 1.2–1.3× to that difference is a conjecture that I could not test without hardware counters. Second, the gap grows to 1.43–1.56× for $n \geq 1536$, which is exactly where `reorder_ikj` loses about 20 % of its throughput while blocked loses at most about 10 %. `reorder_ikj` sweeps the whole of B ($8n^2$ bytes) once per row of C; B is 8 MiB at $n = 1024$ and 18 MiB at $n = 1536$, i.e. it stops fitting in the 12 MiB L3 between the two sizes, whereas a $b \times b$ tile of B (512 KiB for $b = 256$) stays cache-resident. This timing is consistent with H2 (a blocking benefit beyond the L3), but H2 as stated is only partly supported: part of the measured benefit appears at all sizes and is not a cache effect. The block-size study agrees with Lam et al. [6] that performance is sensitive to b and not monotone; here the best tested b was the largest one. Even the best kernel reaches only 4.80 GFLOP/s, far from the vectorised and register-blocked kernels of optimised libraries [2]; the study measures the effect of two optimisations, not the attainable peak.

RQ-B: when does CSR help? At $n = 2048$, CSR SpGEMM was faster than the dense kernel at every tested density up to 50 % and slower from 70 %, for both patterns; it used less memory up to 30 % and more from 50 %. H3 is supported, with a crossover (about 55 %) much higher than the low-density intuition. A simple cost model explains the location: the dense kernel spends 0.44 ns per multiply-add for n^3 multiply-adds,

SpGEMM about 1.4 ns per multiply-add for about d^2n^3 of them, so the predicted crossover is $d^* = \sqrt{t_D/c_S} \approx 55\,\%$ (with t_D, c_S the two per-operation costs), matching the measurement. The crossover is thus set by the ratio of the per-operation costs of the two kernels, not by the percentage of zeros alone: a faster dense kernel (e.g. a vectorised library) would move it to lower densities, a slower one to higher ones. For memory, CSR needs 12 bytes per stored entry against 8 bytes for a dense entry, and for uniform inputs the output is full from $d \approx 5\%$; with $2 \cdot 12dn^2$ bytes of inputs and $12n^2$ of output the SpGEMM storage reaches the dense $24n^2$ at $d = 50\%$, as observed. The process RSS also includes buffers outside the timed region: the dense process holds the COO input buffers (16 bytes per nonzero) while densifying, which explains its larger RSS at $d \geq 70\%$.

Where indexing, conversion and output costs reduce the advantage (H4). The evidence supports H4 and shows three distinct costs. (i) *Conversions* are irrelevant near the crossover but dominate at low density, where they reduce a $10^4\times$ kernel advantage to about $300\times$; if the data arrive dense, the O(n^2) scan of dense→CSR sets a floor of about 11 ms at $n = 2048$. (ii) *Indexing and per-row overheads* make each multiply-add 14 ns at $d = 0.1\%$, ten times its cost at high density; a plausible cause is the $O(n)$ initialisation of the accumulator and the row loops, which do not shrink with d. (iii) *Output fill-in*: at equal multiply-add counts, the uniform pattern produces up to 13.7× more output entries and runs up to 4.42× slower than the banded one, and at $n = 16384$, $d = 1\%$ its output needs 2579 MiB against 145 MiB. The banded pattern is faster at equal density, as H4 predicts, but the gap vanishes once its band saturates ($d \geq 50\%$). The same input density therefore gives very different costs depending on the nonzero structure; density alone does not determine performance in these experiments.

Optimisation vs. storage effects. The two parts measure different mechanisms. In Part A the arithmetic is fixed ($2n^3$) and only the memory-access order changes, yielding up to 12.0×. In Part B the sparse representation changes the amount of work itself (by d^2 for uniform inputs) and the size of the output, yielding more than $10^4\times$ at $d = 0.1\%$ but losing at high density because each sparse operation is about three times more expensive than a dense one.

A. Limitations

All results come from one laptop (i7-9750H, gcc 13.3 `-O2`, one thread) with the `powersave` governor and turbo enabled, so the clock frequency was not fixed; other course benchmarks were serialised by the lock but unlocked background work existed (load average up to 1.3). Repetitions were in-process, so the IQR reflects within-process variability; between processes the same blocked kernel at $n = 2048$ took 3.93 s in the block study, 4.13 s in the size sweep and 3.59 s–3.93 s in Part B (different inputs, same work), i.e. up to 15 %, larger than the reported IQRs. Hardware counters were not available (`perf_event_paranoid= 4`), so cache-related explanations are inferences from timing, sizes and throughput. For $n \geq 3072$ only $k = 3$ repetitions were taken. The block-size optimum lies at the edge of the tested grid. Only two synthetic nonzero patterns at one size ($n = 2048$) were used for the crossover; real matrices (e.g. with power-law row lengths) were not tested. Strassen's algorithm, loop unrolling and vectorisation were not studied, and the SpGEMM output columns are left unsorted, which some consumers would require.

VI. Conclusions

Within the tested scope (one i7-9750H core, gcc 13.3 `-O2`, float64), loop reordering and cache blocking made dense multiplication up to 8.37× and 12.0× faster than the naive i-j-k loop, and let both optimised kernels complete $n = 4096$ within the 600 s budget that the baseline exceeded. The benefit grows with n as the baseline becomes memory-bound; blocking adds about 1.4–1.6× over reordering once B exceeds the last-level cache, plus a smaller gain at all sizes that is not explained by cache reuse. For sparse inputs at $n = 2048$, the own CSR + Gustavson SpGEMM was faster than the best dense kernel up to 50 % density and slower from 70 %, with a crossover near 55 % that a per-operation cost model reproduces; it used less memory up to 30 % density. Conversions, per-row overheads and output fill-in reduce the sparse advantage, and the nonzero pattern changes runtime and output size several-fold at equal density. Unresolved are the hardware-level causes (no counters), the behaviour with block sizes above 256, with real sparse matrices, and with vectorised kernels; the parallel versions of the best dense kernel are the subject of Assignment 3.

Reproducibility Materials

Repository: https://github.com/danirodriguezz/bigdata-assignment02-optimised-sparse (submitted version: git tag `a2-submission`). The archive `Assignment02_RodriguezAlonso_Materials.zip` contains the C sources (`src/c/`), correctness tests (`tests/`), input generator and experiment driver (`scripts/`), configurations (`configs/`), input checksums and sidecars (`data/inputs/`; binary inputs are regenerated deterministically by `make inputs`), raw measurements (`results/raw/`, schema in `results/README.md`), the environment snapshot, and the analysis script that produces every table, figure and number of this report (`analysis/make_all.py`). `make all` rebuilds, tests, regenerates the inputs, reruns the benchmark, the analysis and this PDF.

Use of Generative AI

Claude Code (Anthropic, model Claude Opus 5.5) was used to assist with planning the experiment, drafting and debugging the C and Python code (kernels, benchmark driver, tests, analysis scripts) and drafting the LaTeX text of this report. The author reviewed and executed all code; every measurement comes from the author's runs on the machine of Table I and is recorded in `results/raw/`; every reference was checked against its DOI or official page.

REFERENCES

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017, iSBN 978-0-12-811905-1.

[2] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.

[3] F. G. Gustavson, "Two fast algorithms for sparse matrices: Multiplication and permuted transposition," *ACM Transactions on Mathematical Software*, vol. 4, no. 3, pp. 250–269, 1978.

[4] A. Buluç and J. R. Gilbert, "Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments," *SIAM Journal on Scientific Computing*, vol. 34, no. 4, pp. C170–C191, 2012.

[5] GCC Team, "GCC 12 release series: Changes, new features, and fixes," https://gcc.gnu.org/gcc-12/changes.html, section "General improvements": vectorization enabled at -O2 with the very-cheap cost model. Accessed: 2026-10-29.

[6] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, 1991, pp. 63–74.

[7] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[8] V. Strassen, "Gaussian elimination is not optimal," *Numerische Mathematik*, vol. 13, no. 4, pp. 354–356, 1969.

[9] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: SIAM, 2003.

[10] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*, 2015, pp. 1–12.

[11] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIV)*, 2009, pp. 265–276.

[12] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[13] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[14] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.